

Family Album

Setup and operations guide for the self-hosted trip photo album

This guide explains what the Family Album application is, how to install and run it with Docker, how to configure email, storage and maps, how to bring family members on board, and how to use the day-to-day features such as trips, photo uploads, GPS track imports, themes and sharing. It ends with a development setup, backup advice and troubleshooting.

Contents

- 1. What the application does
- 2. Requirements
- 3. Quick start with Docker
- 4. Configuration reference (.env)
- 5. Signing in and inviting family
- 6. Using the album: trips, uploads, activities, tracks, themes, sharing
- 7. Running behind a reverse proxy
- 8. Backups and upgrades
- 9. Development setup and tests
- 10. Troubleshooting

1. What the application does

Family Album is a private web app for organizing family trip photos. Photos belong to a **trip**, and within a trip to an **activity** (a hike, a bike ride, a boat tour, a meal). Everything can be browsed on a **timeline** grouped by day and on a **map**. GPS tracks from a watch, bike computer or Google location history give activities a route and fitness statistics, and can place photos that have no GPS data of their own.

- **Automatic organization.** Uploaded photos land on the right trip by the date they were taken, and on the right activity by time of day.
- **Any camera or phone.** JPEG, PNG, WebP and iPhone HEIC are accepted. Originals are kept; web-sized WebP versions are generated for display.
- **Correct times.** Camera clocks are reconciled using the EXIF time-zone offset, the GPS position, or the trip's time zone. A one-click fix handles cameras left in the wrong zone.
- **Tracks and stats.** Import GPX, Garmin FIT, or Google Timeline exports. Activities show distance, moving time, elevation, pace or speed, heart rate, cadence, power and calories, with charts linked to the map.
- **Photos placed from tracks.** A photo taken during a hike without GPS is positioned by interpolating along the track.

- **Themes.** Each trip or collection gets a look: lighthouse coast, Scottish highlands, Everglades swamp, desert canyon, alpine, dog park, or classic.
- **Sharing.** Trips are private by default, can be shared by a secret link, or made public.
- **Photo links.** Photos can be marked as the same scene, before/after, panorama parts, or related.
- **Passwordless sign-in.** Family members sign in with an emailed link. An admin invites them.

How it is built

The app is a Next.js server with a PostgreSQL database (with the pgvector extension) and a folder for photos. Background work (processing uploads, transcoding clips, parsing track files, geotagging, descriptions, embeddings, faces) runs in a job queue stored in Postgres. The optional ML sidecar is a small Python service on an internal Docker network that never talks to the internet.

Component	Technology	Role
Web app	Next.js 16, React 19, TypeScript	Pages, uploads, API routes
Database	PostgreSQL 16 with Prisma 7	Trips, activities, photos, tracks, users
Photo storage	Local folder (Docker volume)	Originals plus generated thumbnails
Jobs	pg-boss (in the Postgres database)	Photo processing, track imports, geotagging, transcoding, descriptions, embeddings, faces, nightly purges
Images	sharp, exif, heic-convert	EXIF reading, resizing, HEIC conversion
Video	ffmpeg (in the app image)	Short clips transcoded to MP4 with a poster; longer videos embedded from YouTube
AI descriptions	Anthropic API (optional, two switches)	A title, caption, tags and a search summary per item
ML sidecar	FastAPI, OpenCLIP, MiniLM, InsightFace (optional)	Image and text embeddings, face templates, all on your own server
Maps	MapLibre GL with OpenStreetMap tiles	Clustered photo markers and track lines
Email	Nodemailer over SMTP	Sign-in and invite links

2. Requirements

- A machine that runs **Docker** and **Docker Compose v2**: a home server, NAS, Raspberry Pi 4 or better, or a small VPS. Without the ML sidecar 2 GB of RAM is comfortable; with it (faces, similar photos, semantic search) plan on 4 GB with a swap file, or 8 GB without.
- Disk space for photos. Originals are kept, so budget for the size of your library plus roughly 15% for renditions.
- An **SMTP account** for sending sign-in emails (any provider: Gmail app password, Fastmail, Mailgun, Postmark, your ISP). Optional for testing; links can be read from the log instead.
- Optional: a domain name and a reverse proxy with HTTPS if you want to reach the album from outside your home network.

3. Quick start with Docker

- 1 Get the code onto the server, either by cloning the repository or by copying the project folder.
- 2 Create your configuration file from the template and edit it. At minimum set **ADMIN_EMAIL** to your own address.
- 3 Build and start the stack. The first build takes a few minutes.
- 4 Open the site, enter the admin email, and follow the sign-in link.

```
git clone https://github.com/Macrophage87/PhotoAlbum.git
cd PhotoAlbum
cp .env.example .env
nano .env          # set ADMIN_EMAIL, SMTP_* for real email, a new POSTGRES_PASSWORD, FORGET_KEY
docker compose up --build -d
```

The stack has two services by default. **db** is PostgreSQL 16 with pgvector, its data in the **pgdata** volume. **app** is the web server, with photos in the **photos** volume mounted at /data/photos. On every start the app applies pending database migrations before serving, so upgrades need no manual database step. Three optional services live behind compose profiles: **worker** (background jobs in their own container), **ml** (the local ML sidecar) and **ml-init** (a one-off download of the model weights into the **ml-models** volume).

Browse to **http://localhost:3000** on the server itself. The port is published on the loopback address only (APP_BIND=127.0.0.1), so from other machines the album is reached through an https reverse proxy (section 7), even on a home network. Plain http://<server>:<port> from another device does not work, and APP_BIND=0.0.0.0 does not make it: the app runs in production mode, where the sign-in cookie is marked Secure and browsers keep it only over https (or on localhost), and without a proxy in front anyone can forge the X-Forwarded-For header the sign-in rate limits go by. Change the host port with APP_PORT in .env if 3000 is taken.

Reading the sign-in link without email

If SMTP_HOST is left empty, sign-in and invite links are printed to the container log instead of being sent. This is fine for first setup and for a purely local install.

```
docker compose logs -f app | grep "auth/verify"
```

Loading demo data

A seed script creates two sample trips (a lighthouse-themed week in Acadia, Maine with a hike, its track and stats, and a Scottish Highlands trip), a collection spanning both, a short clip, a YouTube embed, two named people, a pet and AI-style descriptions. It needs no internet access and no API key: the descriptions are copied from a recorded example and the face templates are made-up vectors, so you can see what the features look like before turning anything on.

```
docker compose exec app node_modules/.bin/tsx prisma/seed.ts
```

Useful commands

```
docker compose ps          # service status
docker compose logs -f app  # follow app log
docker compose restart app  # restart the web app
docker compose down        # stop (volumes are kept)
docker compose pull && docker compose up --build -d  # rebuild after updating the code
```

4. Configuration reference (.env)

All settings live in the .env file next to docker-compose.yml. The compose file also reads POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB and APP_PORT from it. DATABASE_URL and PHOTO_STORAGE_ROOT are set by the compose file for the container, so you only need to change them for a non-Docker install.

Variable	Default	Purpose
APP_URL	http://localhost:3000	Public URL of the site. Used in every emailed link and in redirects. Set it to your real address (with https) when behind a proxy.
HSTS_INCLUDE_SUBDOMAINS	false	With an https APP_URL, browsers are told to keep to https for a year. true extends that to every subdomain of the album's hostname; only set it if they all serve https.
ADMIN_EMAIL	you@example.com	The first person to sign in with this address becomes an admin. Allowed to sign in without an invite only while the album has no admin.
SIGN_IN_MAIL_PER_HOUR	200	Protects the mail provider's quota. Each address holds at most three unused links at a time (asking again says to use the newest one); its first always goes out, and only the second and third count against this hourly total.
SMTP_HOST	(empty)	Mail server hostname. Leave empty to log links instead of sending mail.
SMTP_PORT	587	587 for STARTTLS, 465 for implicit TLS.
SMTP_USER / SMTP_PASS	(empty)	Mail server credentials.
SMTP_SECURE	false	Set true when using port 465.
SMTP_FROM	Family Album <album@example.com>	Sender shown in emails. Many providers require it to match the account.
POSTGRES_USER / _PASSWORD / _DB	photoalbum	Database credentials used by both containers. Change the password on an internet-facing host.
APP_PORT	3000	Host port published by Docker.
APP_BIND	127.0.0.1	Address the port is published on. Keep 127.0.0.1 with a reverse proxy on the same host. A proxy on another machine needs an address it can reach, a firewall that really covers the port (Docker's published ports bypass ufw), and to set X-Forwarded-For itself. Never 0.0.0.0 for plain http from other devices: sign-in needs https, and without a proxy the X-Forwarded-For behind the sign-in rate limits can be forged.
STORAGE_DRIVER	local	Storage backend. Only local is implemented; the code has an interface for adding S3 later.
PHOTO_STORAGE_ROOT	/data/photos	Where originals and renditions are written (inside the container).

Variable	Default	Purpose
MAX_UPLOAD_BYTES	104857600 (100 MB)	Largest single photo accepted.
MAX_IMPORT_BYTES	2147483648 (2 GB)	Largest track or Google export file accepted.
RUN_WORKER	true	Run background jobs inside the web process. Set false and start the worker container (docker compose --profile worker up -d) to split them out.
NEXT_PUBLIC_TILE_URL	(empty, uses OpenStreetMap)	Raster tile template for the map, e.g. from MapTiler or Stadia.
NEXT_PUBLIC_MAP_STYLE_URL	(empty)	Full MapLibre style JSON URL. Overrides the tile URL.
NEXT_PUBLIC_MAP_GLYPHS_URL	(empty)	Font glyph URL template for map labels. All NEXT_PUBLIC values are compiled into the browser bundle: rebuild with docker compose up --build after changing them.
MAX_CLIP_SECONDS	90	Longest clip accepted for upload. Longer videos go on YouTube as unlisted and are linked in.
MAX_VIDEO_UPLOAD_BYTES	1073741824 (1 GB)	Largest clip file accepted.
MAX_SCAN_UPLOAD_BYTES	1073741824 (1 GB)	Largest 3D scan (GLB, USDZ, PLY, SPZ) accepted.
YOUTUBE_API_KEY	(empty)	Optional. With a Google Data API key, embedded videos also show their length.
ANNOTATION_ENABLED	false	Operator half of the AI-description switch. The other half is an admin's opt-in on the Admin page.
ANTHROPIC_API_KEY	(empty)	Key for the AI helper. Nothing is sent without it and both switches.
ANNOTATION_MODEL	claude-opus-5	Model used for descriptions; claude-sonnet-5 and claude-haiku-4-5 cost less.
ANNOTATION_QUIET_MINUTES	30	Minutes without edits before an unreviewed item is sent.
ANNOTATION_RAW_RETENTION_DAYS	30	Days the raw AI responses are kept for debugging.
ML_URL / ML_TOKEN	(empty)	Address and shared secret of the local ML sidecar (http://ml:8000 in Docker). Set both or neither.
ML_IDLE_UNLOAD_SECONDS	300	The sidecar frees its models after this much idle time.
FACE_INDEXING_ENABLED	false	Operator half of the face-detection switch; the admin's opt-in is the other half.
FACE_UNNAMED_RETENTION_DAYS	180	Faces nobody names are deleted after this many days.
GEOCODER_ENABLED / GEOCODER_URL	true / OpenStreetMap	Address lookup when setting a photo's place by hand. Only the words typed are sent, from the server.
PET_MATCHING_ENABLED	true	Animal spotting through the ML sidecar. A pet you have tagged once is proposed on later look-alikes.
IMPORT_INBOX_DIR	/data/imports	Folder the Google Takeout importer reads zip files from (the imports volume). Empty hides the section on the Admin page.

Variable	Default	Purpose
GOOGLE_OAUTH_CLIENT_ID / _SECRET	(empty)	OAuth client for the Google Photos picker button. See the deployment guide for the Google Cloud steps.
TOKEN_ENCRYPTION_KEY	(empty)	32 random bytes in base64 (openssl rand -base64 32). Encrypts members' Google tokens; required with the client id.
FORGET_KEY	(empty)	32 random bytes in base64 (openssl rand -base64 32), a different one for each instance. Forgotten people's names are kept only as hashes under a key made from it and a salt in the database. Required in production for forgetting anybody for good. It is not in the database dumps: back it up separately, and never change it, since once somebody has been forgotten, running without it or with another pauses forgetting and the AI helper until it is put back. deploy/update.sh makes one if .env has none.
ANTHROPIC_BASE_URL, YOUTUBE_*_URL	(empty)	Endpoints for test doubles; leave empty.
VISITOR_STATS_ENABLED	true	Count how many pages are opened and by how many browsers, for the Admin page. Counted here, sent nowhere.
VISITOR_STATS_RETENTION_DAYS	90	Days those counts are kept; a nightly job deletes older ones and the salt that hashed them.
CSP_REPORT_ONLY	false	Report Content-Security-Policy violations instead of blocking them, while trying a new map provider.
COMPOSE_PROFILES	(empty)	Read by Docker Compose: set to ml (and/or worker) so every 'up' includes those optional services.

Gmail example. SMTP_HOST=smtp.gmail.com, SMTP_PORT=587, SMTP_SECURE=false, SMTP_USER=your address, SMTP_PASS=an App Password generated in your Google account security settings (normal passwords are rejected), SMTP_FROM=the same address. Restart the app after changing .env: `docker compose up -d`.

5. Signing in and inviting family

There are no passwords. A person types their email on the sign-in page and receives a link that is valid for 15 minutes and can be used once: opening it shows a Sign in button, and pressing that is what signs them in (so a mail scanner that opens links first does not use it up). They then stay signed in on that browser as long as they visit at least once every 12 weeks.

Who can sign in

- The address in ADMIN_EMAIL, while the album has no admin yet. The first sign-in with it creates the admin account; once an admin exists it is an ordinary address, so removing that account sticks.
- Anyone who already has an account.
- Anyone with a pending invite for their address.

- Everyone else sees the same 'check your email' message but no email is sent and no account is created, so the site never reveals who is a member.

Installing on a phone

The album is a progressive web app. Once it is reachable over HTTPS, family members can install it without an app store: on iPhone open the site in Safari, tap Share, then Add to Home Screen; on Android open it in Chrome, tap the menu, then Install app; on a desktop browser click the install icon in the address bar. It then opens full-screen with its own icon, and shows an offline notice rather than a browser error when there is no connection.

Inviting members

- 1 Sign in as the admin and open **Admin** from the top navigation (the /admin page).
- 2 Enter the relative's email address and choose Member or Admin. An invite email is sent (or logged when SMTP is not set up).
- 3 The relative opens the invite link, which sends them a sign-in link for that address. From then on they simply sign in with their email.
- 4 The Admin page lists members and pending invites. From there you can change roles, revoke an invite, or remove a member.

If the only admin can no longer read their email

ADMIN_EMAIL creates an admin only while the album has none, so changing it later does nothing. Fix it in the database instead, from the server: move the admin account to a new address (first command), or make another member an admin (second). Addresses are stored in lower case; **UPDATE 1** means it worked.

```
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" "$POSTGRES_DB" -c "UPDATE \"User\" SET email = '
docker compose exec -T db sh -c 'psql -U "$POSTGRES_USER" "$POSTGRES_DB" -c "UPDATE \"User\" SET role = '1'
```

Members can create trips, upload photos, edit anything, and share trips. Admins can additionally manage members. Every signed-in member sees every trip; the album is a shared family space, not per-user galleries.

Who has been looking

The Admin page also counts pages opened, over the last week, month or three months: how many were family signed in, how many came in on a secret link, and how many were anyone at all on something public — so you can tell whether the link you sent was ever opened. Below that are the trips, collections and photographs people actually look at, the sites that linked them here, and when each member last read anything. It is all counted on this server and sent nowhere. What is written down for each page is the kind of page and the trip, collection or item it was about, never the address bar, never a share link, and never anyone's network address: browsers are told apart by a number made with a salt that is random for the day and deleted with it. Everything is deleted after VISITOR_STATS_RETENTION_DAYS (90 by default), and VISITOR_STATS_ENABLED=false counts nothing at all.

6. Using the album

Creating a trip

Choose **New trip** on the front page. Give it a title, start and end dates, the local time zone of the destination, and a theme. The date range matters: it is how uploaded photos are matched to the trip, and how Google location exports are filtered. The time zone is used when a photo's camera did not record one and when grouping days on the timeline.

Uploading photos

- Use the **Upload** button inside a trip's Photos tab, or the global Upload page. Drag and drop or pick files; several upload at once.
- Each photo is processed in the background: EXIF date, GPS and camera details are read, the image is rotated correctly, and thumbnail and medium WebP renditions are made. Tiles show a spinner until ready.
- A photo uploaded from a trip page is attached to that trip. Uploaded from the global page, it is matched to whichever trip's dates contain the day it was taken. If several trips overlap, it stays unassigned and shows a 'needs trip' badge until you assign it.
- If the photo's time falls inside an activity's window on that trip, it is attached to the activity too.
- The photo detail page shows EXIF data, lets you edit the caption, move the photo to a trip or activity, shift the time zone if the camera was set wrong, set it as the trip cover, and link it to other photos.
- In any gallery, select several photos for bulk actions: assign to an activity, add to a trip or a collection, or delete. Photos that matched no trip wait on the **Photos without a trip** page. Before a change makes photos visible to more people (adding a private trip's photo to a public collection, say) the album says so and asks you to confirm.

iPhone HEIC photos work, but conversion is slow (a few seconds each). If you prefer, set the iPhone camera to 'Most Compatible' so it saves JPEG. Renditions are always WebP, so browsers never need HEIC support.

Collections

A collection gathers photos from any trip, or from none, around a theme: a person, a place, a year, the dog. Create one from the front page, then add photos by ticking the collection on a photo's page or by selecting photos in a gallery. A photo can sit in any number of collections. Each collection has its own cover, theme, order (drag to arrange, or sort by date), timeline and map, and is private, shared by secret link, or public, exactly like a trip. Because a photo is visible to anyone who may open its trip or any collection holding it, putting a private trip's photo into a public collection publishes that photo; the album warns before it happens.

Reviewing uploads and searching

After an upload, choose **Add notes and file them** to open the review screen. Type a note for the whole batch or for a selection ("Grandma Jo's 80th at the lake house, everyone came"), attach items to a trip or a collection, and mark them reviewed. The **Review** page lists everything nobody has reviewed yet. The search box finds photos by caption, note, title, trip or collection name; visitors without an account search only public content.

AI descriptions (optional)

An AI helper (Anthropic's Claude) can write a caption, a description and search tags for each photo, using the notes you typed. It is off until two switches are on: `ANNOTATION_ENABLED` with an API key in the configuration, and an admin's opt-in on the Admin page, which lists exactly what is sent (a reduced copy of the image, your notes, the date and camera, the trip and collection names, and the names of confirmed people whose recognition an admin turned on and who are adults, plus pet names; never the original file or any face data). Each item is sent once after it is reviewed. Any photo, trip or collection can be marked as never to be sent. The Admin page can also describe the existing library in one go, showing the count and an approximate cost first and asking you to type the number to confirm.

Suggestions and the ML helper (optional)

With the local ML sidecar running (see the deployment guide), the review screen suggests which trip or collection each new photo belongs to, with the reason ("taken during the trip, 2 km from the Ocean Path hike"), and search understands meaning as well as words. The sidecar runs entirely on your own server: nothing about a photo leaves it. It needs about 2 GB of memory of its own, so plan on 4 GB with a swap file, or 8 GB.

People (optional)

With the ML sidecar running, an admin can turn on face detection from the Admin page after reading what it does. Faces are then found and grouped on your own server. On the People page, name a group once and it becomes a person with a page of their photos. Whether that person is recognized in new photos is a separate decision only an admin can make, off by default and never for a child without a parent's instruction; faces nobody names are deleted after six months, and anyone can be forgotten at any time. Names are only ever shown to family members. Once someone is recognized, the album asks "Probably Grandma Jo?" on later photos and waits for you to say yes or no. Pets get a record with their species and years; tag one by hand from the photo viewer, and with the sidecar running the album spots animals too and asks "Probably Biscuit?" on later look-alikes of the same kind (a flock record like "the chickens" is proposed whenever chickens are seen).

Bringing photos over from Google Photos

Two routes, neither a background sync (Google no longer allows one). **Pick a few:** once the operator has set up a Google OAuth client (deployment guide), each member presses **Connect Google Photos** on the Upload page once; after that, **Pick from Google Photos** opens Google's own picking page, and what they pick is copied into the album and shown on the review screen. Google leaves the location out of those copies, so add places on the review screen or file them to a trip with a track. **Everything at once:** export your library with Google Takeout (Google Photos only), copy the zip files to the server's import inbox, and import each from the Admin page. Dates, places, descriptions and album membership come across from Google's sidecar files, and each Google album becomes a private collection. A photo already in the album is not imported twice: the export fills in anything it is still missing, so importing the same export again is safe and repairs photos that arrived without a place. Delete the zip from the inbox once its photos are in: it is an unencrypted copy of your export.

Similar photos

With the ML sidecar running, the album keeps track of which photos look alike. The Graph page (members only) draws them as a web of thumbnails you can pan and zoom; color it by trip, collection, person or who uploaded, and drag the slider to show only the closest matches. Each photo page also shows a small strip of look-alikes. Only photos you can see are ever linked.

Videos

Short clips (MP4, MOV or WebM, up to 90 seconds) upload like photos and are converted in the background to a web-playable file with a poster; they play muted when you hover over them in a gallery and with controls in the viewer. A longer file is refused before it uploads.

Longer videos live on YouTube. Upload the video there as **Unlisted**, then choose **Add a YouTube video** on a trip's or collection's Photos tab (or the Upload page) and paste the link. The album fetches the title and poster once and shows the poster in the gallery; the video plays in place only when someone presses play, through YouTube's privacy-enhanced player. Unlisted means anyone with the YouTube link can watch, whatever the album's visibility. A weekly check flags videos that were deleted or made private on the Admin page.

Activities

An activity is anything you did within a trip: hike, bike, run, kayak, drive, boat, sightseeing, food, or other. Create one by hand from the trip's Activities tab with a title, type and time window, or let a FIT or GPX import create it. Photos whose time falls inside the window are attached automatically and shown inside the activity's card on the timeline.

Importing GPS tracks

Open the trip's **Import** page and upload a file. The import runs in the background and reports what it created.

Source	What to upload	Result
Garmin, Wahoo, Strava, Komoot, AllTrails, phone apps	.gpx or .fit file (export the activity from the app or Garmin Connect)	An activity with route, statistics and charts. Sport is detected from FIT files. Photos in its time window are attached and, if they lack GPS, positioned from the track.
Google Maps Timeline (on the phone)	Google Maps > profile picture > Your Timeline > (three dots) > Location and privacy settings > Export Timeline data. Upload the resulting Timeline.json.	One location trace per day of the trip, drawn dashed on the map. No activity is created.
Google Takeout (older exports)	Records.json, or the monthly Semantic Location History JSON files.	Same as above.

Only points inside the trip's dates are imported, so uploading a whole multi-year export is safe; the app streams the file rather than loading it into memory. Statistics computed from a track: distance, moving and elapsed time, elevation gain and loss, average and maximum speed or pace, heart rate, cadence, power and normalized power, and calories. Values recorded by a Garmin device in the FIT file take precedence over computed ones so they match what the watch showed.

Timeline and map

- The trip **Timeline** groups photos and activities by local day, with a day list on the side for jumping around. Photos taken during an activity appear inside its card next to a small route drawing and key stats.
- The trip **Map** shows clustered photo markers and color-coded track lines. Click a cluster to zoom in, a marker to preview and open the photo, a track to open the activity.
- The **activity page** shows the full-size map, the stat grid, and elevation, speed, heart-rate and power charts. Moving the mouse along a chart moves a marker along the route.
- The global **Map** and **Timeline** pages in the top navigation show every trip together, each keeping its own theme colors.

Themes

Pick a theme when creating a trip or later in its Settings. A theme sets the palette, fonts, illustrated header art, map marker shape and decorative motif for that trip's pages, including the shared and public views.

Theme	Feel	Suggested for
Classic	Neutral slate and blue	Anything
Lighthouse Coast	Navy, fog white, red and white stripes, sea glass	Maine, coastal New England, the Pacific coast
Highlands	Heather purple, peat brown, moss green, mist	Scotland, Ireland, moorland
Swamp	Deep green, cypress brown, egret white, sunset orange	Everglades, bayou, wetlands
Desert Canyon	Terracotta, sandstone, sage, turquoise sky	Utah, Arizona, the Southwest
Alpine	Ice blue, granite gray, pine green, snow	The Alps, Rockies, ski trips
Dog Park	Fur brown, meadow green, tennis-ball yellow, paw prints	A collection about the dog

Sharing a trip

Each trip has a visibility setting in its Settings page.

- **Private** (default): only signed-in family members can see it.
- **Anyone with the link**: a secret link is generated. Relatives without accounts can view the photos, activities, timeline and map, but cannot edit. The link can be rotated at any time, which immediately disables the old one.
- **Public**: anyone can browse the trip without signing in and it appears on the front page for visitors.

Privacy. A shared or public trip exposes every photo, activity and location trace on that trip, which reveals where people were and when. Keep photos you would not share on a private trip, and rotate a link if it spreads further than intended.

7. Running behind a reverse proxy

To reach the album from outside your network, put it behind a reverse proxy that terminates HTTPS (Caddy, nginx, Traefik, or your NAS's built-in proxy). Two things need attention:

- Set **APP_URL** in `.env` to the public address, for example `https://album.example.com`, so emailed links and redirects use it.
- Raise the proxy's request body limit. Photos are up to 100 MB by default and Google exports can be far larger. The app-side caps are `MAX_UPLOAD_BYTES` and `MAX_IMPORT_BYTES`.
- The app's port is published on 127.0.0.1 only (`APP_BIND`), which suits a proxy on the same host. A proxy elsewhere needs `APP_BIND` set to an address it can reach, and then a firewall that really covers that port: Docker's published ports bypass `ufw`.

Caddy example (automatic HTTPS):

```
album.example.com {
  reverse_proxy localhost:3000
  request_body {
    max_size 2GB
  }
}
```

nginx example:

```
server {
  listen 443 ssl;
  server_name album.example.com;
  client_max_body_size 2g;
  proxy_read_timeout 600s;
  location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
```

When exposed to the internet, also change `POSTGRES_PASSWORD` from the default, and consider not publishing the database port at all (the default compose file does not).

8. Backups and upgrades

Two things hold all your data: the **pgdata** volume (database) and the **photos** volume (originals and renditions). Back up both. The photos volume is the irreplaceable one.

```
# database dump
# pipefail so a failed pg_dump is an error rather than a tiny .gz; the db container supplies its own user
set -o pipefail
# umask in a subshell: the dump is readable by you only, and later commands are not affected
(umask 077 && docker compose exec -T db sh -c 'exec pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" | gzip > a
# prints 1 when the dump is complete (pg_dump writes this line last); 0 means it stopped partway
gzip -cd album-db-$(date +%F).sql.gz | tail -n 20 | grep -c 'PostgreSQL database dump complete'

# photos (copy the volume contents)
docker run --rm -v photoalbum_photos:/data -v $(pwd):/backup alpine \
  tar czf /backup/album-photos-$(date +%F).tgz -C /data .
```

The volume name is prefixed with the compose project name, normally the folder name; check with `docker volume ls`. Restoring is the reverse: create the volumes, extract the archive into the photos volume, and pipe the dump into `psql` on the db container.

Upgrading

```
git pull
docker compose up --build -d          # add --profile ml if you run the sidecar and COMPOSE_PROFILES is n
docker compose run --rm ml-init       # only after an upgrade that changes the sidecar's models
```

Database migrations run automatically at container start. Take a database dump before upgrading, as a precaution. An AI-description backfill still in progress is cut short by an upgrade; the Admin page says so under that run within about an hour. Wait until none of its rows reads in progress, then start the backfill again for what is left (the app refuses a new run while one is open). The first upgrade to the media-hub release replaces the database container with the `pgvector` image; the data in the `pgdata` volume is kept as it is.

Upgrading from a release without `FORGET_KEY`. Before the first upgrade to the release that brought it, add `FORGET_KEY=` with the output of `openssl rand -base64 32` to `.env` where it has none (**never replace a key that already exists**), a different key for each instance (staging and live never share one), and back it up separately from the database dumps. `deploy/update.sh` makes one when `.env` has none, but an older checkout's first deploy runs the older script, which does not; until there is a key nobody can be forgotten for good. If the album has already forgotten somebody and the key is lost, never make a new one: put the original back (the deployment guide, step 10).

The app's port is now published on 127.0.0.1 only (`APP_BIND`). If other devices used to open the album as `http://<server>:<port>`, put an https reverse proxy in front instead (section 7); `APP_BIND=0.0.0.0` does not bring plain http back. Signing in needs https, since the session cookie is marked Secure, and without a proxy anyone can forge the X-Forwarded-For header the sign-in rate limits go by.

9. Development setup and tests

For working on the code, run the app on your machine with Node 22 and pnpm, and use Docker only for Postgres and a local mail catcher. The database must have the `pgvector` extension, which the compose image includes and a stock PostgreSQL package does not; with your own server, install `pgvector` as well. Only a superuser may create the extension (it is not marked trusted), so either make the app's role a superuser, or have a superuser run `CREATE EXTENSION vector` once in `template1`, so every database created afterwards has it, including the `_test` and `_e2e` ones the test scripts create. A database that already exists needs the same command run in it.

The storage paths in `.env.example` (`/data/photos`, `/data/imports`) are container paths a normal user cannot write to. Point `PHOTO_STORAGE_ROOT` and `IMPORT_INBOX_DIR` at folders of your own, such as `./data/photos` and `./data/imports`, or leave `IMPORT_INBOX_DIR` empty.

```
pnpm install
cp .env.example .env          # DATABASE_URL at localhost:5432; storage paths to your own folders
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d db mailpit
pnpm prisma migrate dev       # create the schema
pnpm db:seed                  # optional demo data
pnpm dev                      # http://localhost:3000
```

Mailpit catches all outgoing mail. Set SMTP_HOST=localhost and SMTP_PORT=1025 in .env and read sign-in emails at http://localhost:8025. Without SMTP settings the links are printed in the terminal running pnpm dev.

Scripts

Command	What it does
pnpm dev / pnpm build / pnpm start	Development server / production build / run the build
pnpm lint / pnpm typecheck	ESLint and TypeScript checks
pnpm test	Vitest unit tests (parsers, stats, timeline, auth). Migrates, and creates if needed, a photoalbum_test database before running.
pnpm build && pnpm test:e2e	Playwright smoke tests (sign-in, upload, sharing) against a photoalbum_e2e database
pnpm db:migrate / pnpm db:deploy / pnpm db:seed	Create a migration / apply migrations / load demo data
pnpm worker	Run the background job worker as its own process (with RUN_WORKER=false on the web process). In Docker use the worker compose profile instead.
node scripts/make-fixtures.mjs	Regenerate test fixture files (GPX, FIT, Google JSON, photos)

Code layout

src/app	routes (Next.js App Router), including /api handlers
src/components	UI components (photos, timeline, map, charts, admin)
src/lib/auth	magic links, sessions, trip access rules
src/lib/images	EXIF, renditions, HEIC conversion
src/lib/tracks	GPX / FIT / Google parsers, stats, simplification, storage encoding
src/lib/jobs	pg-boss queue and every background job
src/lib/annotation	AI descriptions (request building, pricing, apply)
src/lib/people	consent rules, face clustering and matching
src/lib/search	keyword plus semantic search
src/lib/graph	similar-photo edges and the graph page data
src/lib/ml	client for the sidecar
src/themes	theme registry and per-theme art
ml	the Python ML sidecar
prisma	schema, migrations and the offline seed
tests	unit tests, fixtures, Playwright specs

A GitHub Actions workflow runs lint, typecheck, unit tests and a Docker build on every push.

10. Troubleshooting

Symptom	Likely cause and fix
Sign-in says 'check your email' but nothing arrives for a new person	Only ADMIN_EMAIL, existing members and invited addresses receive a link; everyone else gets the same message and no email. Invite them from the Admin page, and check .env spelling (restart the app after edits).

Symptom	Likely cause and fix
No sign-in email arrives	SMTP_HOST empty (links are in the log: <code>docker compose logs app grep auth/verify</code>), wrong port/SECURE combination, or the provider rejects SMTP_FROM. SMTP errors are logged: check <code>docker compose logs app</code> . Test with Mailpit first.
Links in emails point to localhost	APP_URL is still the default. Set it to the public URL and restart.
Uploads fail around 1 MB or 100 MB	Reverse proxy body limit (raise <code>client_max_body_size</code> / <code>max_size</code>) or <code>MAX_UPLOAD_BYTES</code> .
Photo stays on the spinner	Processing job failed; see the app log. A failed photo shows its error on the detail page. HEIC files take several seconds each.
Photo landed on the wrong day or trip	Camera clock or time zone. Open the photo and use the time-zone shift control, or set the trip's time zone correctly before uploading.
Photo has no location on the map	Photos uploaded from an Android phone arrive without one: the phone removes the position from any copy it hands to another app, which is why the phone's own gallery still shows it on a map. Import that library from Google Takeout (the position is in its sidecar files) and the photos already in the album are filled in. Otherwise: import a GPX/FIT/Google trace for that day, or open the photo and use Set a place, which also works for a whole gallery selection at once.
Google import created nothing	No points inside the trip's dates. Check the trip dates and that the file is a Timeline export (Timeline.json, Records.json or Semantic Location History).
Takeout import skipped everything	The photos are already in the album (same bytes or the same Google item). The import report on the Admin page shows the count.
Deleting a Takeout archive says the inbox is not writable	The imports volume was made by an older image and belongs to root. Run once: <code>docker compose run --rm --user root --entrypoint chown app -R node:node /data/imports</code>
"Google Photos needs to be connected again"	Google revoked the member's grant (a password change, or six months unused on a test-mode consent screen). Press Connect again on the Upload page.
No "Probably Biscuit?" for a pet	Spotting needs the sidecar, <code>PET_MATCHING_ENABLED</code> , and one hand tag of that pet on a photo where an animal was found; the Admin page shows the sighting counts.
Map is blank	Tile server unreachable or <code>NEXT_PUBLIC_TILE_URL</code> / <code>NEXT_PUBLIC_MAP_STYLE_URL</code> wrong. Clear them to fall back to OpenStreetMap. <code>NEXT_PUBLIC</code> values are baked in at build time, so rebuild after changing them.
Container restarts on start	Migration failure or database not ready; run <code>docker compose logs app</code> . The db service has a health check so app waits for it.
Disk full	Photos volume. Move Docker's data root or mount a larger disk at the volume; originals are never deleted automatically.